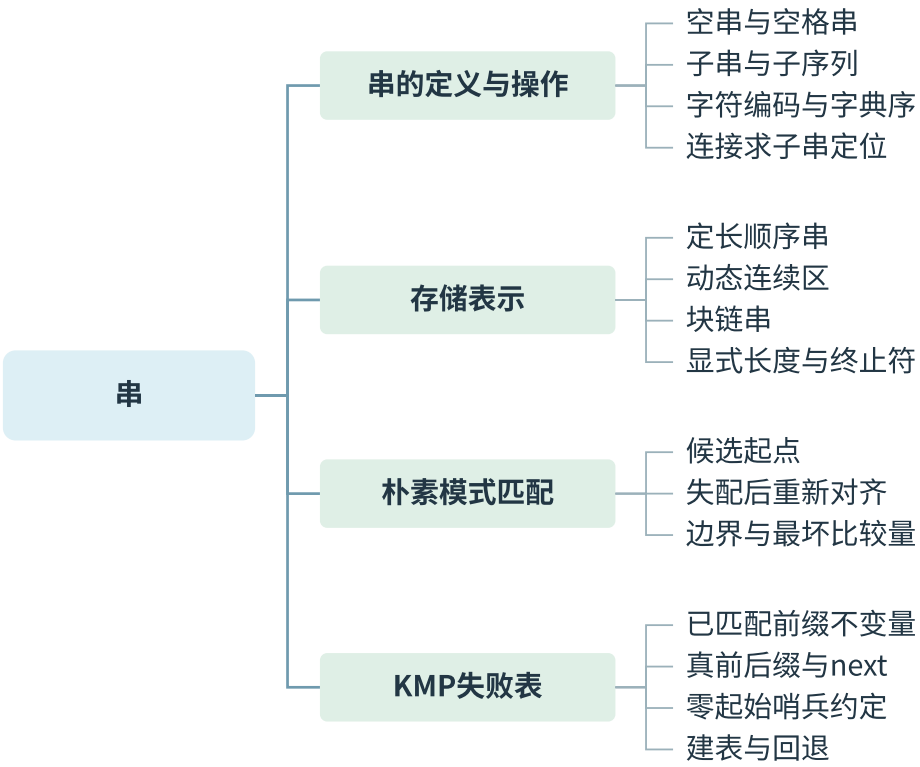
4 串

知识导图



最小反例·空格仍是字符

空串长度为 0，一个空格构成的串长度为 1。仅凭打印后看起来空白无法判断串是否为空；接口判空应检查长度或表示规则，而不是观察屏幕上的可见字形。

知识导图（续）



错项排除·一次候选失败才移起点

朴素法在某候选起点内逐字符比较，失配后起点才加 1；主串和模式的工作指针可以回退。不能把朴素法的“重试下一个起点”与 KMP 保持 i 不动混为同一规则。

模式匹配要在主串中寻找一段连续字符，使它与模式串逐字符相同。朴素方法失败后重试下一个起点；KMP 利用已经相等的部分决定怎样移动模式串。二者处理的是同一个匹配问题，差别在于是否重复比较已经确认相等的字符。

不变量·回退保留相等边界

失配时已知 T[i−j..i−1] 等于 P[0..j−1]。新候选只可保留旧段中同时是模式前缀的后缀；next[j] 给出最长可保留长度，主串当前字符尚未被丢弃。

4.1 串、子串与下标

串是由零个或多个字符组成的有限序列，串长为字符个数。长度为 0 的是空串；只含空格的串仍含字符，不是空串。**子串**必须由原串中连续的一段字符构成，允许不连续选取字符得到的是子序列。两个串相等，须长度相等且每个对应字符相等。

本章令主串为 $T[0..n-1]$，模式串为 $P[0..m-1]$，匹配位置返回从 0 开始的下标。题目若使用从 1 开始的位序，最终位置加 1，但中间 `next` 数组还涉及失败哨兵的约定，不能只把表中某一列机械加 1。空模式是否匹配应由接口约定；以下算法把空模式匹配位置约定为 0，主要讲解 $m>0$ 的情形。

串可用定长数组、动态分配的连续存储区或块链存储。C 语言中以 `\0` 结束的字符串是一种具体表示；结束符本身不计入通常的串长。使用显式长度的串也可以包含值为 0 的字符。求子串、连接、替换与模式匹配是不同运算，不能把“在主串中找到与模式串相同的一段字符”称为串连接。

4.2 朴素匹配怎样回退

先把模式首字符与主串起点 $s=0$ 对齐，从头逐一比较。若某处不同，则把起点改为 $s+1$，重新从模式首字符比较。若模式全部相等，返回 s；若对每个满足 $s \le n-m$ 的起点都失败，则不存在匹配。

```
naive(T, n, P, m):
    for s=0 to n-m:
        j=0
        while j<m and T[s+j]==P[j]: j=j+1
        if j==m: return s
    return -1
```

当 $m>n$ 时，没有候选起点，直接失败。一次尝试最多比较 m 次，总共最多 $n-m+1$ 个起点，所以最坏时间为 $O((n-m+1)m)$，常简记为 $O(nm)$；辅助空间为 $O(1)$。

最坏情况来自“前面大段都相同，到末尾才失败”。例如主串连续很多个 a，模式为 `aaaab`。每次对齐都要重新比较一段 a，而这些比较并不能排除紧接着的下一个起点。若每次第一个字符就失配，比较次数则近似候选起点数。不能把最坏上界当成每次的实际比较次数。

4.3 KMP 的 next 为什么只依赖模式串

设当前已经匹配 j 个字符，即 $T[i-j..i-1]=P[0..j-1]$，现在 $T[i] \ne P[j]$。主串这段已知内容就是模式的前 j 个字符。要让模式右移后仍保留尽可能多的已匹配字符，新的模式前缀必须等于旧匹配段的后缀。

因此，要在 $P[0..j-1]$ 中找最长的一对相等真前缀与真后缀，设其长度为 k。真前缀和真后缀都不允许取整个串。右移后，旧段最后 k 个

字符与新模式前 k 个字符已经相等，下一步只需比较 $T[i]$ 与 $P[k]$。主串指针 i 保持不变，模式指针从 j 退到 k，模式整体右移了 $j-k$ 位。

错项排除 · next 不含当前失配字符

本章 next[j] 研究 P[0..j−1] 的真前后缀，而不是 P[0..j]。例中失配下标 5 之前是 abaab，最长边界 ab 长 2；把 c 算进去会让表错位。

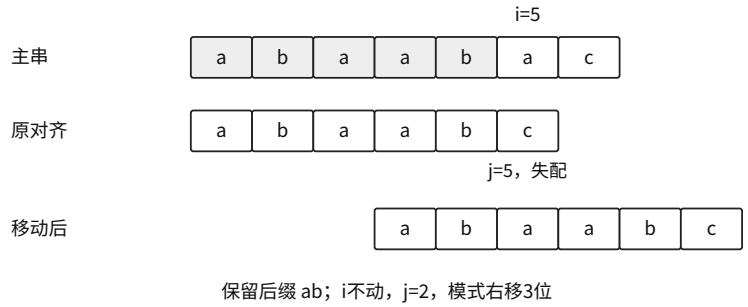


Figure 4.1: 已匹配段 abaab 的后缀 ab 与模式前缀 ab 对齐，主串当前字符保持原位

4.3.1 本书的 next 定义

对 $j \geq 1$，`next[j]` 等于 $P[0..j-1]$ 的最长相等真前后缀长度；令 `next[0]=-1` 作哨兵。`j=0` 已失配时，模式没有可保留的前缀，应把主串推进一位再从模式首字符开始；代码借 `j=-1` 统一完成这个动作。

以 `P=abaabc` 为例：

下标 j	0	1	2	3	4	5
P[j]	a	b	a	a	b	c
next[j]	−1	0	0	1	1	2

求 `next[5]` 时，考察的是位置 5 之前的 `abaab`，不把 `P[5]=c` 算进去。`abaab` 的最长相等真前后缀是 `ab`，长度 2，所以失败后去 `P[2]` 继续比较。若把 `next` 误算成“包含当前位置的前后缀长度”，就会把表错开一格。

```
buildNext(P, m):
    next[0]=-1
    i=0; k=-1
    while i<m-1:
        if k==-1 or P[i]==P[k]:
            i=i+1; k=k+1
            next[i]=k
        else:
            k=next[k]
```

建表函数的前提是 $m > 0$；空模式直接返回匹配位置 0，不调用建表函数。求表时同样是在模式内部做匹配：已知长为 k 的相等前后缀，若下一字符也相等，就扩展一位；不等时，退到这段前缀自己的更短边界继续尝试。不能每次失配都直接把 k 清零，否则会丢掉仍可复用的边界。

```
kmp(T, n, P, m, next):
    if m==0: return 0
    i=0; j=0
    while i<n and j<m:
        if j==-1 or T[i]==P[j]:
            i=i+1; j=j+1
        else:
            j=next[j]
```

```
    if j==m: return i-m
    return -1
```

主串指针只前进，不回退；模式指针可回退。每次字符相等时二者同时增加，失败时只有 j 减少；j 总减少量受此前总增加量约束。因此匹配扫描为 $O(n)$，求表为 $O(m)$，总时间 $O(n+m)$，辅助数组占 $O(m)$。“每个主串字符只能比较一次”并不成立：同一个主串字符可能与模式的多个位置比较，只是总比较量仍为线性。

不变量·只跳过必然失败的位置

若 P[next[j]] 与已经失配的 P[j] 相同，那么同一主串字符与它再比较仍失败。nextval 压缩这类失败链；只有获得这一字符相等条件，才能证明跳过不会漏解。

4.3.2 例 4-1　第一次失配后哪个指针改变

例 4.3.1 (配套例题)

配套例题：字符串 s 为 `abaabaabacacaabaabcc`，模式串 t 为 `abaabc`。采用 KMP 匹配，第一次出现失配 `s[i]≠t[j]` 时，`i=j=5`，下次开始匹配时，i 和 j 分别是（　）。

A. i=1，j=0　B. i=5，j=0　C. i=5，j=2　D. i=6，j=2

解析。由“i=j=5 时第一次失配”可知本题使用从 0 开始的下标：前面已匹配 `abaab`，当前主串字符 a 与模式字符 c 不同。已匹配段的后缀 `ab` 可与模式前缀 `ab` 重合，因此 `j=next[5]=2`，i 仍为 5，答案为 **C**。此后比较的是同一个主串 a 与模式第 3 个字符 a；这一步若相等，才把 i、j 同时加 1。

若使用从 1 开始的 `next[1]=0` 约定，相同位置对应的指针是 `i=j=6`，失败后 `j=3`。它与从 0 开始的“5 退到 2”表达同一次对齐，不能把表的哨兵 0 和字符下标 0 当成同一个值。

4.3.3 nextval 省掉哪些一定失败的比较

若当前 `P[j]` 已与主串字符失配，而 `P[next[j]]==P[j]`，退到该位置再比较仍必然失败。可以继续沿失败链接退到不同字符，或直接退到哨兵。把这种合并后的失败位置存入 `nextval`。

在本章 0 起始约定下，先求普通 `next`，再从小到大求：

```
nextval[0]=-1
for j=1 to m-1:
    k=next[j]
    if k>=0 and P[j]==P[k]: nextval[j]=nextval[k]
    else: nextval[j]=k
```

这项优化只跳过“退过去仍因字符相同而失败”的比较，并不能任意加大右移距离；过度右移会漏掉合法匹配。某些实现把前缀函数记为 `pi` 或 `lps`，其第 j 项包含当前字符。读题时先确认定义，再换算表，不能仅凭数组名断定语义。

4.3.4 例 4-2　修正后最长能移动几位

例 4.3.2 (配套例题)

配套例题：KMP 使用修正后的 next 数组进行模式匹配，模式串 S=`aabaab`。当主串中某字符与 S 中某字符失配时，S 向右滑动的最长距离是（　）。

A. 5　B. 4　C. 3　D. 2

解析。按本章定义，逐位置求表并计算 `j-nextval[j]`：

j	0	1	2	3	4	5
S[j]	a	a	b	a	a	b
next[j]	−1	0	1	0	1	2
nextval[j]	−1	−1	1	−1	−1	1
右移距离	1	2	1	4	5	4

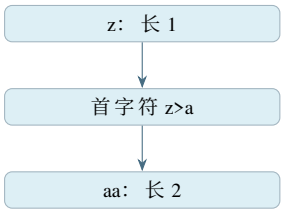
最长距离为 5，答案为 **A**。例如在 j=4 的 a 处失配，普通失败链接退到 j=1 的 a，再到 j=0 的 a，都会与当前主串字符重复失配；修正表直接到 −1。下一轮主串也前进一位，于是新对齐起点相对旧起点实际前进 5 位。

4.3.5 比较次数要与实际实现对应

普通 next 与 nextval 的字符比较次数可能不同；程序里对 `j==-1` 的判断不是一次字符比较。若题目要求“单个字符之间的比较次数”，只记录真正执行 `T[i]==P[j]` 的次数。一次匹配成功后是否继续查重叠出现，也取决于任务：只找首次出现可立即返回；要找全部出现，须保留末尾可重用边界，不能一律把模式指针清零并跳过已读位置。

最小反例·长串未必大

按通常英文编码的字典序，z 大于 aa，虽然前者只有一个字符。第一次不同字符就决定比较结果；只有短串完全等于长串前缀时，串长才参与最后判断。



字典序先看首个不同字符，z 大于 aa

4.4 字符编码、串比较与基本操作

定义 4.4.1 (字符序列与表示单位)

串的逻辑元素是字符，串长是所约定字符单位的个数。编码把字符映射到计算机表示；比较、求长和取子串必须使用一致的单位。408 中的英文字符示例通常按单字节处理，不能据此把任意 UTF-8 文本的字节数直接当作用户看到的字符数。

串比较通常采用字典序：从首字符起寻找第一个不同位置，按该字符的编码序比较；若较短串的全部字符都等于较长串的对应前缀，则短串较小。长度不同不意味着长串一定大，例如按通常英文编码序，`z` 大于 `aa`。比较返回值一般只约定负、零、正，不保证恰为 −1、0、1。

赋值、判空、求长、清空、连接、求子串、串比较和定位是不同操作。连接把两串首尾接起来；求子串提取一个连续区间；定位返回模式首次出现的位置。复制输出区时还要考虑容量与源/目标是否重叠，不能只改元数据长度而不复制内容。

例 4.4.1 (自拟：区分空串、空格串与前缀关系)

设 A 为空串，B 包含一个空格，C=`ab`，D=`abc`。哪些是空串？按字典序比较 C 与 D，并说明 D 中连续子串 `bc` 的位置。

只有 A 长度为 0；B 含一个字符。C 是 D 的真前缀，故 C 小于 D。`bc` 在 D 中的起始下标为 1，若题目用从 1 开始的位序则为 2。空串可视为任意串的子串，但接口仍须约定定位返回值；本章返回首次合法起点 0。

4.5 定长、动态与块链表示

课程连 接 · 编码单位决定下

字符编码把符号落实为存 UTF-8储。的一个字符可能占多个字节，字节下标不总是字符位置；本章英文题按单字节模型分析，换输入模型时先确定元素单

定长顺序串预分配最大容量，并另存实际长度；超容量时必须定义失败或截断行为。截断会改变运算结果，不能将它悄悄当作完整连接成功。堆分配顺序串按实际需要申请连续区，长度或容量改变时可能重新申请和复制，旧地址不保证保持有效。

块链串让一个结点保存多个字符，减少每字符一个指针的开销；末块若未用满，需要有效长度或明确的填补规则。链式表示适合某些局部接入操作，却不能因此宣称按字符下标访问为常数时间。C 风格以终止符`\0`判结束的字符串和“数组 + 显式长度”的串是不同表示；前者不能把中途的`\0`作为普通有效字符继续处理，后者可以。

表示	如何知道长度	主要边界	随机访问
定长数组 + 长度	显式 length	最大容量、结果截断	字符下标 $O(1)$
动态连续区 + 长度	显式 length	申请失败、扩容复制	字符下标 $O(1)$
块链存储	总长度及末块信息	跨块定位、未用字符位	通常需沿块查找
C 终止符串	扫描到`\0`	终止符与有效容量	已知下标可访问，求长需扫描

4.6 求子串的安全边界与实现

> **区间先统一成起点加长度**
>
> 用 0 起始下标 pos 与长度 len 表示半开区间 $[pos, pos+len)$。合法条件为 $0 \le pos \le n$、$0 \le len \le n - pos$；pos=n 只允许取空子串。与检查 pos+len 不越界相比，检查 len≤n−pos 还可避免实现中加法溢出。

```cpp
#include <string>
bool subString(const std::string& s, int pos, int len,
               std::string& result) {
    if (pos < 0 || len < 0) return false;
    const std::size_t first = static_cast<std::size_t>(pos);
    const std::size_t count = static_cast<std::size_t>(len);
    if (first > s.size() || count > s.size() - first)
        return false;
    result = s.substr(first, count);
    return true;
}
```

这里用标准字符串演示逻辑操作和边界，元素仍按字节处理。求长和下标的表示单位必须一致；失败时不修改 result，成功提取空串则会把 result 置空，两种结果不能只靠返回串内容区分。

4.7 KMP 的可执行实现与线性界

以下 C++17 实现与正文的 0 起始、`next[0]=-1`一致，前提是主串和模式串长度均可用 int 表示，且单个元素采用单字节比较。空模式单独成功，不访问 next[0]。

```cpp
#include <vector>
```

```cpp
std::vector<int> buildNext(const std::string& p) {
    const int m = static_cast<int>(p.size());
    std::vector<int> next(m, -1);    if (m == 0) return next;
    int i = 0, k = -1;
    while (i + 1 < m) {
        if (k == -1 || p[i] == p[k]) {
            ++i; ++k;
            next[i] = k;
        } else {
            k = next[k];
        }
    }
    return next;
}
```

```cpp
int kmpFirst(const std::string& text, const std::string& p) {
    const int n = static_cast<int>(text.size());
    const int m = static_cast<int>(p.size());
    if (m == 0) return 0;
    if (m > n) return -1;
    const auto next = buildNext(p);
    int i = 0, j = 0;
    while (i < n && j < m) {
        if (j == -1 || text[i] == p[j]) {
            ++i; ++j;
        } else {
            j = next[j];
        }
    }
    return j == m ? i - m : -1;
}
```

允许重复比较，为何总量仍是线性

每次成功匹配或哨兵推进都会使 i 增加，发生次数最多 n；其余迭代只令 j 严格下降。j 的增加总量受上述推进次数限制，而 j 最低为 -1，因此全部下降量也为 $O(n)$，下降迭代数不会超过该量。两类迭代合计 $O(n)$。建表时将 i、k 作同样分析，i 最多推进 m-1 次，k 的回退受其此前增长限制，故预处理为 $O(m)$，总时间 $O(n+m)$。

□

本证明没有声称每个主串字符只比较一次。例如同一个 `text[i]` 可在 j 依次回退时与多个模式字符比较；真正受控的是总的推进与回退数量。计算字符比较次数时，`j==-1` 这样的整数判断不是字符比较；不同 next/nextval 实现也可能有不同具体次数。

4.8 由历史错题检查 KMP 的单调量

配套做题本第 4 章 4.2 第 2 题问 “指示主串的指针” 的特点。标准 KMP 中主串指针可以前进，也可以在模式回退时保持原位，但不会变小。因此应选 “不会变小”，而不是把 “可以前进、可以不动” 误解成 “可以回退”。模式指针才沿 next 失败链接回退，二者角色必须分清。

4.9 主动回忆自测

以下为原创补充，用于检查失败表定义及边界。

1. 对模式 `aaaa`，按本章约定写普通 next 与 nextval。

错项排除·不会变小才是准确表述

标准 KMP 中 i 在匹配成功或哨兵推进时变大，在模式回退时不动。因此 “不会变大” 错误，“可能变小” 也错误；主串指针的单调不减与模式指针的失败回退必须分开看。

2. 某主串字符与模式 `abab` 的最后一个 b 失配，普通 next 应退到哪里？主串指针是否前进？
3. 主串长 n、模式长 m，只执行一次 KMP 匹配但预先已有可用 next 表，扫描部分复杂度是多少？

答案。1. next 为 `-1,0,1,2`，nextval 为 `-1,-1,-1,-1`。2. `next[3]=1`，先退到模式下标 1，主串指针不动；若使用 nextval，因两个位置都是 b，可直接退到 0 继续。3. $O(n)$；总算法连同求表才记为 $O(n+m)$，表的存储仍要计入空间开销。

不变量·半开区间分清空与越界

从 pos 取 len 个字符表示半开区间 [pos,pos+len)。pos 等于 n 时只可取长度 0；检查 len 不超过 n−pos，既表明边界，又避免先计算 pos+len 可能产生的整数溢出。

4.9.1 补充自拟回忆题

按导图解释对象、表示与失败转移

4. 字典序中 `ab` 与 `b` 谁大？是否只按串长比较？
5. 求子串 pos=n、len=0 合法吗？pos=n、len=1 呢？
6. 长为 m 的模式比主串长时，为何可以直接返回失败？空模式为什么要另行约定？
7. 本章 next[j] 是否包含 P[j]？若把它当作包含当前位置的前缀函数会错在哪里？
8. nextval 把多个同字符失败位置跳过，为什么仍不遗漏合法匹配？

补充题核对答案

4. b 大于 ab，因为第一处 a 与 b 已决定大小；长度只在较短串完全是较长串前缀时决定次序。
5. 前者合法，表示末端空子串；后者越界。
6. 非空模式必须落在主串的一个等长连续区间，m>n 时没有候选起点；空模式不需要字符比较，接口须约定返回值，本章返回 0。
7. 不包含 P[j]，只研究 P[0..j−1]。当作包含当前位置的前缀函数，会使失败表语义错一格，改变回退位置。
8. 已知当前主串字符不等于 P[j]；若更短候选位置仍是同一字符，比较必然失败，可继续回退。只跳过能证明失败的位置，不是任意加大位移。